

Relatório - Implementação de Algoritmos de Ordenação

1. Implementação dos Algoritmos

Foram implementados seis algoritmos clássicos de ordenação em linguagem C: Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort e Shell Sort. Cada algoritmo foi desenvolvido em função separada, garantindo organização e modularidade no código. Os algoritmos Bubble, Selection e Insertion possuem complexidade média $O(n^2)$. Já Quick Sort e Merge Sort utilizam a estratégia de divisão e conquista, apresentando complexidade média $O(n \log n)$. O Shell Sort é uma melhoria do Insertion Sort, utilizando intervalos (gaps) para melhorar o desempenho.

2. Resultados Obtidos nos Testes

Foram realizados testes com diferentes tipos de arrays: - Array aleatório - Array ordenado crescente - Array ordenado decrescente - Array quase ordenado. Os testes foram executados com diferentes tamanhos (ex: 10, 1000 e 10000 elementos), utilizando a função `clock()` da biblioteca para medir o tempo de execução em milissegundos. Observou-se que os algoritmos quadráticos apresentaram aumento significativo no tempo conforme o tamanho do array crescia.

3. Comparação de Desempenho

Nos testes com arrays grandes (ex: 10000 elementos), os algoritmos Quick Sort e Merge Sort apresentaram os melhores tempos de execução. O Bubble Sort e Selection Sort foram os mais lentos, principalmente em arrays grandes e ordenados de forma decrescente. O Insertion Sort apresentou bom desempenho em arrays quase ordenados, sendo eficiente nesse cenário específico. O Shell Sort apresentou desempenho intermediário, sendo mais rápido que os algoritmos quadráticos tradicionais, mas geralmente inferior ao Quick e Merge em grandes volumes de dados.

Conclusão

Conclui-se que a escolha do algoritmo de ordenação depende do cenário e do tamanho do conjunto de dados. Para grandes volumes, algoritmos $O(n \log n)$ são mais indicados. Para conjuntos pequenos ou quase ordenados, algoritmos simples como Insertion Sort podem ser suficientes. A atividade permitiu compreender na prática o impacto da complexidade algorítmica no desempenho computacional.